

Aula 4 (Módulo 06) — Projetos: IOC Extractor e Hash Checker

Fechando o módulo com dois projetos que você vai reutilizar bastante: extrair indicadores de um texto qualquer, e verificar a "identidade" de um arquivo pelo hash. Código completo em [src/ioc-extractor/](#) e [src/hash-checker/](#).

Parte 1 — IOC Extractor

O problema que resolve

Relatórios de threat intel, e-mails de phishing reportados, notas de investigação — tudo isso é **texto não estruturado**. Extrair manualmente cada IP, domínio e hash mencionado é lento e sujeito a erro. O objetivo aqui é automatizar exatamente essa extração.

As regex por categoria

```
PATTERNS = {
    "ips": re.compile(r"\b\d{1,3}\.\d{1,3}\.\d{1,3}\.\d{1,3}\b"),
    "dominios": re.compile(
        r"\b(?:[a-zA-Z0-9](?:[a-zA-Z0-9-]{0,61}[a-zA-Z0-9])?\.\.+"
        r"(?:com|net|org|io|gov|edu|br|ru|cn|info|biz|xyz)\b",
        re.IGNORECASE,
    ),
    "emails": re.compile(r"\b[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}\b"),
    "md5": re.compile(r"\b[a-fA-F0-9]{32}\b"),
    "sha1": re.compile(r"\b[a-fA-F0-9]{40}\b"),
    "sha256": re.compile(r"\b[a-fA-F0-9]{64}\b"),
}
```

Ponto importante: os três padrões de hash (`md5` / `sha1` / `sha256`) **não se confundem entre si** porque o `\b` (limite de palavra) força o comprimento exato — uma string de 64 caracteres hexadecimais nunca casa com o padrão de 32, porque sobriam 32 caracteres hex depois do `\b` de fechamento, o que quebraria o limite de palavra.

`\b` (word boundary) — por que ele importa tanto aqui

Sem `\b`, o padrão de IP (`\d{1,3}\.\d{1,3}\.\d{1,3}\.\d{1,3}`) poderia casar parcialmente dentro de um número maior por acidente. `\b` garante que o padrão comece e termine numa **borda de palavra** (início/fim do texto, ou ao lado de um espaço/pontuação) — evitando falsos positivos.

Separando IP público de privado

```
IP_PRIVADO_RE = re.compile(r"^(10\.|192\.168\.|172\.(1[6-9]|2\d|3[0-1])\.)")
```

Isso usa exatamente as faixas de IP privado da RFC 1918 que vimos no Módulo 03, Aula 1 — mas escrito como regex, incluindo a parte mais sutil (`172.16.0.0` até `172.31.255.255` — daí o `(1[6-9]|2\d|3[0-1])` para cobrir só a faixa

16-31 do segundo octeto).

Por que importa para segurança: em um relatório de IOC, IPs **privados** geralmente representam a **própria rede da vítima** (o host comprometido, não o atacante) — então vale a pena separá-los logo na extração, para não misturar "indicador do atacante" com "identificação da vítima".

Rodando

```
cd src/ioc-extractor
python3 extractor.py exemplo_relatorio.txt --json iocs.json
```

Parte 2 — Hash Checker

Por que hash importa (revisão rápida)

Um hash é uma "impressão digital" de um arquivo — dois arquivos com o mesmo conteúdo, byte a byte, sempre produzem o mesmo hash; qualquer mudança, por menor que seja, muda o hash completamente. Isso permite **identificar um arquivo sem ambiguidade**, sem depender do nome (que um atacante pode trocar facilmente).

Lendo o arquivo em blocos (por que não `arquivo.read()` direto)

```
CHUNK_SIZE = 65536 # 64 KB

with open(caminho, "rb") as arquivo:
    while True:
        bloco = arquivo.read(CHUNK_SIZE)
        if not bloco:
            break
        md5.update(bloco)
        sha1.update(bloco)
        sha256.update(bloco)
```

Se o arquivo for muito grande (ex.: alguns gigabytes), carregar tudo de uma vez com `arquivo.read()` consumiria memória demais. Ler em **blocos** (`CHUNK_SIZE` de cada vez) e ir **atualizando** o objeto de hash incrementalmente (`.update()`) resolve isso — o resultado final é matematicamente idêntico a calcular sobre o arquivo inteiro de uma vez.

Note também o modo `"rb"` (em vez de `"r"`, que vimos na Aula 2) — `"b"` significa **binário**: arquivos executáveis, imagens etc. não são texto, e abrir em modo texto corromperia os dados.

Comparando com hash conhecido ou lista

```
cd src/hash-checker
python3 hash_checker.py arquivo_exemplo.txt --esperado <hash>
python3 hash_checker.py arquivo_exemplo.txt --lista hashes_maliciosos.txt
```

Por que importa para segurança: esse é o fluxo básico de qualquer checagem contra Threat Intelligence (Módulo 12) — você tem um arquivo suspeito, calcula o hash, e verifica se ele já é **conhecido** como malicioso (seja comparando com um hash específico de um relatório, seja contra uma lista maior). Ferramentas comerciais fazem a mesma coisa, só que consultando bancos de dados online enormes em vez de uma lista local — o princípio é idêntico.

3. Conectando o Módulo 06 inteiro

Aula 1 (variáveis/condições/loops/funções) → vocabulário básico
Aula 2 (arquivos/regex/JSON/CSV) → ferramentas de manipulação de dados
Aula 3 (parser de log SSH) → projeto: DETECTAR padrão em log
Aula 4 (IOC extractor + hash checker) → projetos: EXTRAIR indicadores e VERIFICAR identidade de arquivo

Os três projetos juntos (`ssh-brute-force-parser` , `ioc-extractor` , `hash-checker`) já são peças reais de portfólio — cada um resolve um problema concreto de Blue Team, com código testado e documentado.

4. Exercício prático

Pegue o relatório de exemplo (`exemplo_relatorio.txt`) e:

1. Rode o `ioc-extractor` nele.
2. Escolha um dos domínios extraídos e pesquise manualmente (sem ferramenta automatizada) se parece com **typosquatting** de alguma marca conhecida (lembra do Módulo 03, Aula 4?).
3. Escreva, num arquivo `reports/exercicio-ioc-extractor.md` , um mini-resumo dos IOCs encontrados, como se fosse parte de um relatório de incidente real.

5. Recapitulando o Módulo 06

- `\b` (word boundary) evita falsos positivos em regex, especialmente importante para diferenciar hashes de tamanhos diferentes.
- Separar IP público de privado logo na extração ajuda a distinguir "indicador do atacante" de "identificação da vítima".
- Ler arquivos grandes em blocos (`.update()` incremental) evita estourar memória — importante para hash de arquivos reais.
- Modo `"rb"` é obrigatório para arquivos binários (executáveis, imagens) — modo texto corromperia os dados.

Módulo 06 concluído. 🎉

Próximo módulo

Módulo 07 — Git e GitHub (portfólio): agora que temos projetos reais no repositório, formalizamos como usar Git/GitHub profissionalmente — commits, branches, pull requests — para continuar documentando esse portfólio.

Fontes recomendadas

- **Python — documentação oficial do módulo `hashLib`**: referência completa de MD5/SHA1/SHA256 em Python. <https://docs.python.org/3/library/hashlib.html>
- **Python — "Regular Expression HOWTO"**: guia oficial aprofundado de regex em Python, incluindo grupos nomeados e word boundaries. <https://docs.python.org/3/howto/regex.html>
- **RFC 1918 — "Address Allocation for Private Internets"**: já citada no Módulo 03, Aula 1 — base da regex de IP privado usada aqui. <https://www.rfc-editor.org/rfc/rfc1918>